

LLM

COMPLETE BANGLA GUIDE · সম্পূর্ণ বাংলা গাইড

LLM সম্পূর্ণ বাংলা বই

Large Language Models —
প্রাথমিক ধারণা থেকে ফাইন-টিউনিং পর্যন্ত

ট্রান্সফর্মার · প্রম্পট ইঞ্জিনিয়ারিং · ফাইন-টিউনিং
LoRA · RLHF · DPO · RAG · ডিপ্লয়মেন্ট

১৫+ অধ্যায়

কোড উদাহরণ

বেসিক → অ্যাডভান্স

Python & PyTorch

২০২৫ সংস্করণ

সূচিপত্র

অংশ ১ — ভিত্তি (FOUNDATION)

০১	কৃত্রিম বুদ্ধিমত্তা ও LLM — প্রাথমিক ধারণা	বেসিক
০২	ট্রান্সফর্মার আর্কিটেকচার — কীভাবে LLM কাজ করে	বেসিক
০৩	টোকেনাইজেশন ও Embedding	বেসিক
০৪	প্রম্পট ইঞ্জিনিয়ারিং — কার্যকর প্রম্পট লেখার কলা	বেসিক

অংশ ২ — ফাইন-টিউনিং (FINE-TUNING)

০৫	ফাইন-টিউনিং কী এবং কেন প্রয়োজন?	মধ্যবর্তী
০৬	ডেটাসেট প্রস্তুতি ও পরিষ্কার করা	মধ্যবর্তী
০৭	Supervised Fine-Tuning (SFT) — হাতে-কলমে	মধ্যবর্তী
০৮	LoRA ও QLoRA — কম সম্পদে ফাইন-টিউনিং	মধ্যবর্তী
০৯	RLHF ও DPO — মানুষের পছন্দ থেকে শেখা	মধ্যবর্তী

অংশ ৩ — অ্যাডভান্স (ADVANCED)

১০	RAG — Retrieval Augmented Generation	অ্যাডভান্স
১১	মডেল মূল্যায়ন ও বেঞ্চমার্ক	অ্যাডভান্স
১২	ডিল্লয়মেন্ট ও প্রোডাকশন সার্ভিং	অ্যাডভান্স
১৩	অ্যাডভান্স কৌশল — Merging, Unsloth, MoE	অ্যাডভান্স
১৪	বাংলা AI — বাংলাদেশ প্রেক্ষাপটে LLM	অ্যাডভান্স
১৫	সম্পূর্ণ প্রজেক্ট — বাংলা চ্যাটবট তৈরি	অ্যাডভান্স

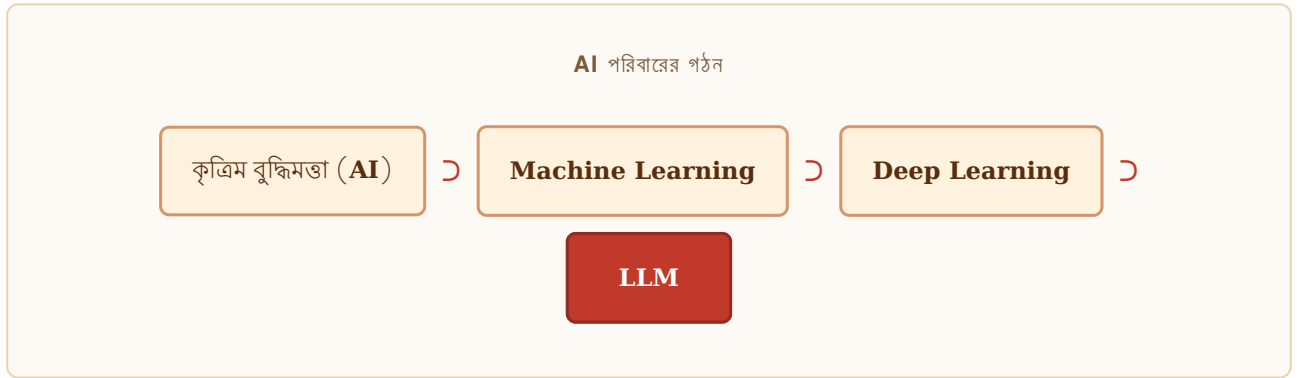
কৃত্রিম বুদ্ধিমত্তা ও LLM

বেসিক

AI পরিবারের মধ্যে LLM-এর অবস্থান এবং মূল ধারণা

AI পরিবার — একটি মানচিত্র

কৃত্রিম বুদ্ধিমত্তা (AI) হলো এমন একটি বিজ্ঞান যা কম্পিউটারকে মানুষের মতো চিন্তা করার ক্ষমতা দেয়। এই বিশাল ক্ষেত্রের মধ্যে Machine Learning, Deep Learning এবং Natural Language Processing (NLP) রয়েছে। LLM হলো এই পরিবারের সবচেয়ে আধুনিক সদস্য।



LLM কী?

Large Language Model (LLM) হলো এক ধরনের Deep Learning মডেল যা বিশাল পরিমাণ টেক্সট ডেটা থেকে ভাষার নিয়মকানুন, তথ্য এবং যুক্তি শিখে নেয়। এটি মানুষের মতো স্বাভাবিক ভাষায় কথা বলতে, লিখতে, অনুবাদ করতে এবং প্রশ্নের উত্তর দিতে পারে।

"LLM হলো এমন একটি যন্ত্র যা ইন্টারনেটের বিশাল জ্ঞানভাণ্ডার থেকে শিখে মানুষের ভাষায় যোগাযোগ করতে পারে।"

বিখ্যাত LLM মডেলসমূহ

মডেল	নির্মাতা	বৈশিষ্ট্য	প্রকার
GPT-4o	OpenAI	মাল্টিমডাল, অনেক শক্তিশালী	বাণিজ্যিক
Claude 3.5	Anthropic	নিরাপদ, দীর্ঘ context	বাণিজ্যিক
Gemini Ultra	Google	মাল্টিমডাল, গণনায় পারদর্শী	বাণিজ্যিক
LLaMA 3	Meta	ওপেন-সোর্স, বিনামূল্যে	মুক্ত
Mistral 7B	Mistral AI	ছোট কিন্তু দক্ষ	মুক্ত
Qwen 2.5	Alibaba	বহুভাষিক, শক্তিশালী	মুক্ত

LLM কীভাবে "শেখে"?

LLM প্রশিক্ষণের মূল লক্ষ্য হলো "পরবর্তী শব্দ অনুমান করা" (Next Token Prediction)। মডেলকে অগণিত বাক্য দেখানো হয় এবং সে শেখে — একটি শব্দের পরে কোন শব্দ আসার সম্ভাবনা বেশি।

উদাহরণ

ইনপুট: "আমি বাংলাদেশে _ _ _"

মডেল অনুমান করে: "খাকি" (৬৫%), "জন্মেছি" (১৫%), "পড়াশুনা করি" (১০%) ...

LLM-এর ক্ষমতা ও সীমাবদ্ধতা

LLM যা পারে

- ▶ স্বাভাবিক ভাষায় কথা বলা
- ▶ অনুবাদ ও সারাংশ তৈরি
- ▶ কোড লেখা ও ডিবাগ
- ▶ সৃজনশীল লেখালেখি
- ▶ প্রশ্নের উত্তর দেওয়া
- ▶ ডেটা বিশ্লেষণ সহায়তা

LLM যা পারে না

- ▶ সত্যিকার "বোঝা" বা অনুভব করা
- ▶ সর্বদা সঠিক তথ্য দেওয়া
- ▶ সাম্প্রতিক ঘটনা জানা (cutoff)
- ▶ ব্যক্তিগত তথ্য মনে রাখা
- ▶ গণনায় সবসময় নির্ভুল
- ▶ ছবি দেখা (text-only মডেল)

HALLUCINATION সমস্যা

LLM মাঝে মাঝে আত্মবিশ্বাসের সাথে ভুল তথ্য তৈরি করে — এটিকে "Hallucination" বলে। যেমন: অস্তিত্বহীন বইয়ের রেফারেন্স দেওয়া বা ভুল তারিখ বলা। সর্বদা গুরুত্বপূর্ণ তথ্য যাচাই করুন।

ট্রান্সফর্মার আর্কিটেকচার

বেসিক

সকল আধুনিক LLM-এর হৃদয় — Self-Attention ও Transformer

ট্রান্সফর্মার কেন বিপ্লব এনেছে?

২০১৭ সালের আগে NLP-তে RNN এবং LSTM ব্যবহার হতো। কিন্তু এগুলো ধীর ছিল এবং দীর্ঘ বাক্যে দুর্বল ছিল। ২০১৭ সালে Google-এর গবেষকরা "Attention is All You Need" পেপারে ট্রান্সফর্মার আর্কিটেকচার প্রবর্তন করেন। এটি NLP জগতে বিপ্লব এনে দেয়।

2017

ট্রান্সফর্মার আবিষ্কার

8

মূল লেখক সংখ্যা

100K+

উদ্ধৃতি সংখ্যা

GPT→

সূচনা করেছে

Self-Attention — মূল রহস্য

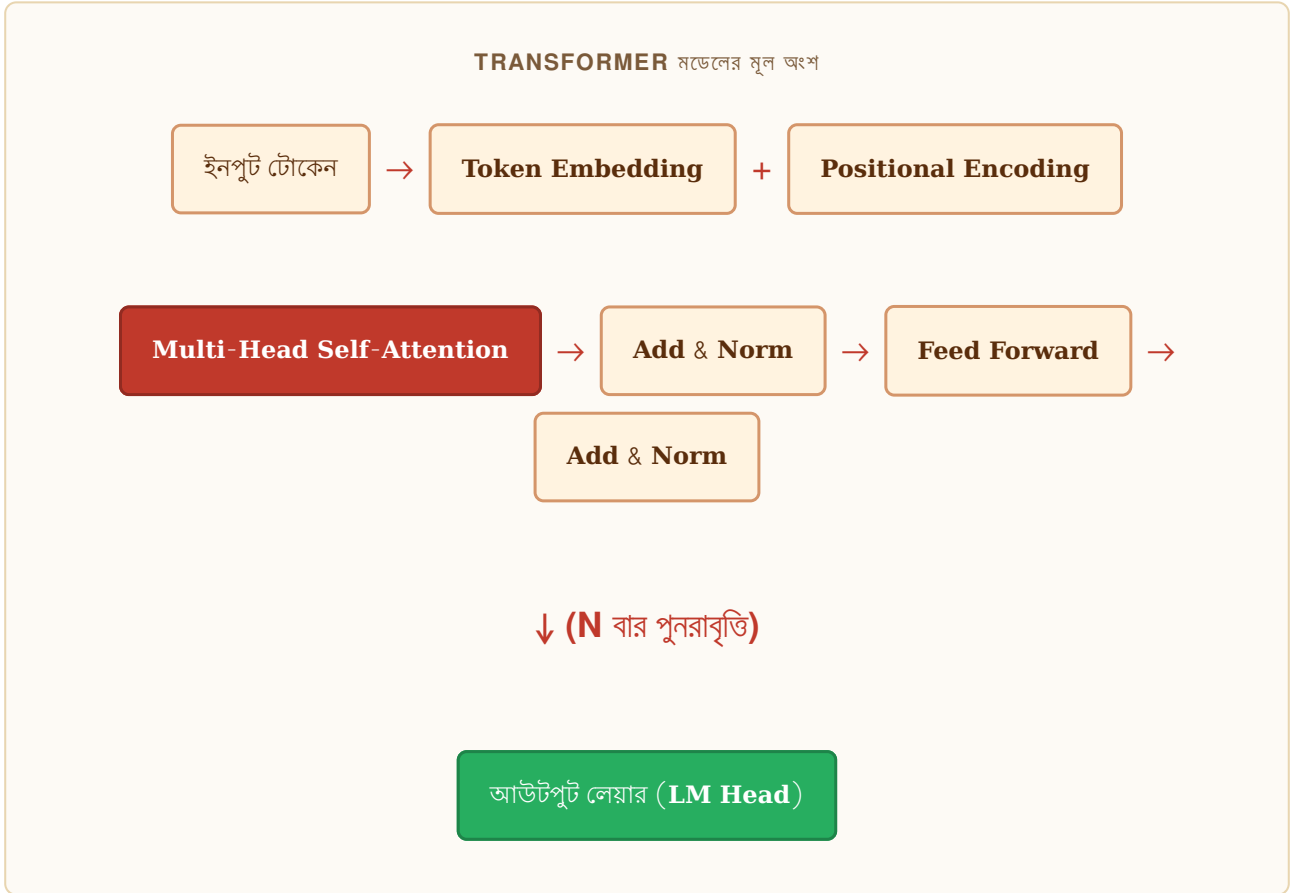
ট্রান্সফর্মারের সবচেয়ে গুরুত্বপূর্ণ অংশ হলো **Self-Attention**। এটি একটি বাক্যের প্রতিটি শব্দকে অন্য সব শব্দের সাথে সম্পর্কিত করে বোঝার চেষ্টা করে।

উদাহরণ দিয়ে **SELF-ATTENTION**

বাক্য: "রাহেলা বাজারে গেল, সে আম কিনল।"

এখানে "সে" বলতে "রাহেলা"কে বোঝাচ্ছে। Self-Attention ঠিক এই সংযোগটি খুঁজে বের করে — "সে"-র সাথে "রাহেলা"-র উচ্চ attention score থাকে।

ট্রান্সফর্মারের গঠন



গুরুত্বপূর্ণ ধারণাসমূহ

Attention Head

- ▶ প্রতিটি head আলাদা প্যাটার্ন শেখে
- ▶ কেউ ব্যাকরণ, কেউ অর্থ
- ▶ GPT-3: ৯৬টি head
- ▶ একসাথে "Multi-Head Attention"

Feed Forward Layer

- ▶ Attention-এর পরের স্তর
- ▶ তথ্য প্রক্রিয়া ও রূপান্তর
- ▶ Activation function: ReLU / GELU
- ▶ মডেলের "স্মৃতি" সংরক্ষণ

Context Window

Context Window হলো মডেল একসাথে কতটুকু টেক্সট "দেখতে" পারে। এটি টোকেনে মাপা হয়। বড় context window মানে মডেল দীর্ঘ কথোপকথন বা বড় নথি মনে রাখতে পারে।

মডেল	Context Window	প্রায় কত পৃষ্ঠা
GPT-3.5	16K tokens	~ 12 পৃষ্ঠা
GPT-4o	128K tokens	~96 পৃষ্ঠা
Claude 3.5	200K tokens	~ 150 পৃষ্ঠা
Gemini 1.5 Pro	1M tokens	~750 পৃষ্ঠা
LLaMA 3.1	128K tokens	~96 পৃষ্ঠা

মনে রাখুন

ট্রান্সফর্মার আর্কিটেকচার বোঝাটা ফাইন-টিউনিংয়ের জন্য অপরিহার্য নয়, তবে LoRA ও Attention-ভিত্তিক পদ্ধতি বুঝতে সাহায্য করে।

টোকেনাইজেশন ও Embedding

বেসিক

ভাষা থেকে সংখ্যায় — মডেল কীভাবে টেক্সট প্রক্রিয়া করে

টোকেনাইজেশন কী?

কম্পিউটার সরাসরি বাংলা বা ইংরেজি বোঝে না — সে শুধু সংখ্যা বোঝে। তাই প্রথমে টেক্সটকে ছোট ছোট খণ্ডে ভাগ করতে হয় যাকে **টোকেন** বলে, তারপর প্রতিটি টোকেনকে একটি সংখ্যায় রূপান্তর করা হয়।

টোকেন উদাহরণ

বাক্য: "আমি পাইথন শিখছি"

টোকেন: ["আমি", "পাইথ", "ন", "শিখ", "ছি"] → [1542, 8734, 892, 5621, 3011]

বাংলা শব্দ ইংরেজির তুলনায় বেশি টোকেনে বিভক্ত হয়, কারণ বেশিরভাগ LLM মূলত ইংরেজিতে প্রশিক্ষিত।

জনপ্রিয় টোকেনাইজার

BPE

- ▶ Byte Pair Encoding
- ▶ GPT মডেলে ব্যবহৃত
- ▶ সবচেয়ে প্রচলিত
- ▶ ঘন ঘন আসা জোড়া মেলায়

WordPiece

- ▶ BERT-এ ব্যবহৃত
- ▶ শব্দের উপসর্গ রাখে
- ▶ "playing" → "play" + "#ing"
- ▶ Likelihood maximize করে

SentencePiece

- ▶ Google উদ্ভাবিত
- ▶ ভাষা-নিরপেক্ষ
- ▶ LLaMA, Gemma-তে
- ▶ বাংলার জন্য ভালো

Embedding — অর্থকে সংখ্যায় রূপ দেওয়া

প্রতিটি টোকেনকে একটি উচ্চমাত্রিক সংখ্যার ভেক্টরে রূপান্তর করা হয়, যাকে **Embedding** বলে। এই ভেক্টরে শব্দের অর্থগত সম্পর্ক এনকোড থাকে।

EMBEDDING-এর জাদু

কিং (রাজা) - ম্যান (পুরুষ) + উইমেন (মহিলা) = কুইন (রানী)

এটি সংখ্যার ভেক্টরের মাধ্যমে বাস্তবে কাজ করে! অর্থাৎ Embedding অর্থের সম্পর্ক ধরে রাখতে পারে।

Python দিয়ে টোকেনাইজেশন

```
tokenization_example.py

from transformers import AutoTokenizer

# LLaMA-3 টোকেনাইজার লোড করুন
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-3.2-1B")

# বাংলা টেক্সট টোকেনাইজ করুন
text = "আমি বাংলাদেশে থাকি "
tokens = tokenizer.encode(text)
print(f"টোকেন সংখ্যা: {len(tokens)}")
print(f"টোকেন ID: {tokens}")

# টোকেনগুলো দেখুন
decoded_tokens = [tokenizer.decode([t]) for t in tokens]
print(f"টোকেন: {decoded_tokens}")

# ইংরেজির সাথে তুলনা
eng_text = "I live in Bangladesh."
eng_tokens = tokenizer.encode(eng_text)
print(f"বাংলা: {len(tokens)} টোকেন, ইংরেজি: {len(eng_tokens)} টোকেন")
```

বাংলা টোকেন সমস্যা

বেশিরভাগ LLM মূলত ইংরেজিতে প্রশিক্ষিত, তাই বাংলা টেক্সট ইংরেজির তুলনায় ৩-৫ গুণ বেশি টোকেন নেয়। এর মানে: বাংলায় একটি ৫০০ শব্দের প্রশ্নের জন্য ইংরেজির চেয়ে অনেক বেশি খরচ ও সময় লাগে।

প্রম্পট ইঞ্জিনিয়ারিং

বেসিক

মডেলের সাথে কার্যকরভাবে যোগাযোগের শিল্প

প্রম্পট ইঞ্জিনিয়ারিং কেন গুরুত্বপূর্ণ?

একই মডেলকে ভালো প্রম্পট দিলে দুর্দান্ত ফলাফল পাওয়া যায়, খারাপ প্রম্পটে খারাপ ফলাফল পাওয়া যায়।
ফাইন-টিউনিং না করেও ভালো প্রম্পটে মডেলের পারফরম্যান্স ৪০–৬০% পর্যন্ত উন্নত হতে পারে।

মূল প্রম্পটিং কৌশল

Zero-Shot Prompting

কোনো উদাহরণ ছাড়া সরাসরি কাজ বলে দেওয়া।

zero_shot.txt

```
# খারাপ প্রম্পট:
বাংলাদেশ প্রম্পটের বনো

# ভালো প্রম্পট (Zero-Shot):
আপনি একজন ভূগোল বিশেষজ্ঞ। বাংলাদেশের ভৌগোলিক
অবস্থান, অয়তন, এবং প্রধান নদীগুলো প্রম্পটের
এটি ব্লকে পয়েন্টে সংক্ষেপে বলুন।
```

Few-Shot Prompting

কয়েকটি উদাহরণ দিয়ে মডেলকে প্যাটার্ন শেখানো।

few_shot.txt

```
# Few-Shot উদাহরণ – অনুভূতি বিশ্লেষণ:
বাক্য: "খাবারটা অসাধারণ ছিল" → ইতিবাচক
বাক্য: "মেরা খুবই বাজে" → নেতিবাচক
বাক্য: "ঠিকঠাক, কিছু ভালো কিছু খারাপ" → মিশ্র
বাক্য: "এত প্রচুর পরিবেশ আগে কখনো দেখিনি" → ???
```

Chain-of-Thought (CoT) Prompting

মডেলকে ধাপে ধাপে চিন্তা করতে বলা — জটিল সমস্যায় অনেক কার্যকর।

chain_of_thought.txt

```
# CoT প্রম্পট:  
একটি দোকানে ৫টি কলম ছিল। মালিক ১২টি কিনলেন,  
তারপর ৭টি বিক্রি করলেন। এখন কতটি আছে?  
  
ধাপে ধাপে চিন্তা করুন:  
ধাপ ১: মোট কলম = ৫ + ১২ = ১৭টি  
ধাপ ২: বিক্রির পরে = ১৭ - ৭ = ১০টি  
উত্তর: ১০টি কলম আছে
```

System Prompt — মডেলের ব্যক্তিত্ব নির্ধারণ

system_prompt_example.py

```
import anthropic  
  
client = anthropic.Anthropic()  
  
response = client.messages.create(  
    model="claude-sonnet-4-20250514",  
    max_tokens=1024,  
    system="\"আপনি বাংলাদেশের একজন অভিজ্ঞ কৃষি বিশেষজ্ঞ  
আপনি মহজ বাংলায় কৃষকদের পরামর্শ দেন।  
সবসময় ব্যবহারিক ও বাস্তবসম্মত পরামর্শ দিন।\"",  
    messages=[  
        {"role": "user", "content": "ধানের ক্লাস্ট রোগ থেকে কীভাবে বাঁচাব?"}  
    ]  
)  
print(response.content[0].text)
```

প্রম্পট ইঞ্জিনিয়ারিং সেবা অভ্যাস

- স্পষ্ট ভূমিকা দিন: "আপনি একজন ডাক্তার / শিক্ষক / কোড রিভিউয়ার..."
- আউটপুট ফরম্যাট বলুন: "৩টি বুলেট পয়েন্টে", "JSON ফরম্যাটে", "১০০ শব্দে"
- প্রসঙ্গ দিন: "আমার শ্রোতা হলো ৬ষ্ঠ শ্রেণির ছাত্র"
- উদাহরণ দিন: কাঙ্ক্ষিত আউটপুটের নমুনা দেখান

- নেতিবাচক নির্দেশনা: "প্রযুক্তিগত শব্দ ব্যবহার করবেন না"

ফাইন-টিউনিং কী এবং কেন?

Pre-training থেকে Fine-tuning — কখন কোনটি প্রয়োজন

ফাইন-টিউনিং — সংক্ষিপ্ত সংজ্ঞা

একটি প্রি-ট্রেন্ড LLM-কে নির্দিষ্ট কাজ বা ডোমেইনের জন্য আরও দক্ষ করে তোলার প্রক্রিয়াই হলো ফাইন-টিউনিং। এটি একটি সাধারণ ডাক্তারকে বিশেষজ্ঞ হার্ট সার্জন বানানোর মতো।

প্রি-ট্রেনিং

- ▶ ট্রিলিয়ন টোকেন ডেটা
- ▶ মাস বা বছর সময়
- ▶ কোটি ডলার খরচ
- ▶ হাজার হাজার GPU
- ▶ সাধারণ জ্ঞান
- ▶ Google, Meta, OpenAI করে

ফাইন-টিউনিং (আমরা করি)

- ▶ হাজার থেকে লক্ষ উদাহরণ
- ▶ কয়েক ঘণ্টা থেকে দিন
- ▶ কয়েক ডলার থেকে শত ডলার
- ▶ একটি বা কয়েকটি GPU
- ▶ বিশেষ দক্ষতা
- ▶ যে কেউ করতে পারে

কখন ফাইন-টিউনিং দরকার?

ফাইন-টিউনিং করুন যখন

- মডেলকে নির্দিষ্ট ফরম্যাটে উত্তর দিতে হবে (JSON, রিপোর্ট, কোড)
- ডোমেইন-নির্দিষ্ট ভাষা শেখাতে হবে (মেডিকেল, আইনি, বাংলা)
- মডেলের "ব্যক্তিত্ব" বা টোন ঠিক করতে হবে
- কম latency দরকার (ছোট ফাইন-টিউনড মডেল দ্রুত)
- ডেটা গোপন রাখতে হবে (নিজের সার্ভারে)
- খরচ বাঁচাতে ছোট মডেল ব্যবহার করতে হবে

ফাইন-টিউনিং না করাই ভালো যখন

- প্রম্পট ইঞ্জিনিয়ারিংয়েই কাজ হয়
- ডেটা খুব কম (১০০-র কম উদাহরণ)
- ঘন ঘন নতুন তথ্য যোগ করতে হয় (RAG ভালো)
- দ্রুত প্রোটোটাইপ দরকার

ফাইন-টিউনিং পদ্ধতির তুলনা

পদ্ধতি	খরচ	মান	কঠিনতা	ব্যবহার
SFT (Full)	বেশি	সর্বোচ্চ	কঠিন	সম্পূর্ণ মডেল পরিবর্তন
LoRA	কম	উচ্চ	মধ্যম	সর্বাধিক প্রচলিত
QLoRA	খুব কম	উচ্চ	মধ্যম	কম GPU-তে
DPO	কম	উচ্চ	মধ্যম	alignment/behavior
RLHF	সর্বাধিক	সর্বোচ্চ	খুব কঠিন	GPT-4, Claude স্তর

ডেটাসেট প্রস্তুতি

সংগ্রহ, পরিষ্কার ও ফরম্যাটিং — সাফল্যের ৭০%

ডেটার গুরুত্ব

"আপনার মডেলের মান আপনার ডেটার মানের বাইরে যেতে পারে না।"

ফাইন-টিউনিংয়ের সাফল্যের ৭০% নির্ভর করে ডেটার মানের উপর। খারাপ ডেটা দিয়ে যত ভালো অ্যালগরিদমই ব্যবহার করুন, ফলাফল ভালো হবে না।

ডেটা ফরম্যাট

Alpaca ফরম্যাট (সহজ)

```
alpaca_format.jsonl

{
  "instruction": "নিচের ইংরেজি বাক্যটি বাংলায় অনুবাদ করুন",
  "input": "The weather is beautiful today.",
  "output": "আজকের আবহাওয়া অসাধারণ সুন্দর।"
}
{
  "instruction": "একটি পাইথন ফাংশন লিখুন যা দুটি সংখ্যা যোগ করে",
  "input": "",
  "output": "def add(a, b):\n    return a + b"
}
```


ChatML / Conversation ফরম্যাট (উন্নত)

```
chatml_format.jsonl

{
  "messages": [
    {
      "role": "system",
      "content": "তুমি বাংলাদেশের একজন সহায়ক AI সহকারী।"
    },
    {
      "role": "user",
      "content": "ঢাকার জনসংখ্যা কত?"
    },
    {
      "role": "assistant",
      "content": "ঢাকার জনসংখ্যা প্রায় ২ কোটি ২০ লক্ষ..."
    }
  ]
}
```

ডেটা পরিষ্কার করা

```
data_cleaning.py

import json
import re
from datasets import Dataset

def clean_text(text: str) -> str:
    # অতিরিক্ত স্পেস সরান
    text = re.sub(r'\s+', ' ', text).strip()
    # HTML ট্যাগ সরান
    text = re.sub(r'<[^>]+>', '', text)
    return text

def is_valid_example(example: dict) -> bool:
    output = example.get('output', '')
    instruction = example.get('instruction', '')

    # মানের ফিল্টার
    if len(output) < 20: return False      # খুব ছোট
    if len(output) > 3000: return False    # খুব বড়
    if len(instruction) < 5: return False  # নির্দেশনা নেই
    return True

# ডেটা লোড ও পরিষ্কার করুন
with open('raw_data.jsonl') as f:
    data = [json.loads(line) for line in f]

cleaned = []
for ex in data:
    if is_valid_example(ex):
        ex['instruction'] = clean_text(ex['instruction'])
        ex['output'] = clean_text(ex['output'])
        cleaned.append(ex)

print(f"মূল: {len(data)}, পরিষ্কার: {len(cleaned)}")
dataset = Dataset.from_list(cleaned)
dataset.save_to_disk("./clean_dataset")
```

বাংলা ডেটাসেট উৎস

নাম	বিষয়	আকার	লাইসেন্স
CC-100 bn	সাধারণ বাংলা	~27GB	মুক্ত
IndicCorp v2	বহুভাষিক	~8GB বাংলা	মুক্ত
BanglaSenti	মতামত বিশ্লেষণ	10K+	গবেষণা
BanglaParaphrase	বাক্য পুনর্লিখন	50K+	গবেষণা
BanglaQA	প্রশ্নোত্তর	~30K	গবেষণা

ডেটা কতটুকু দরকার ?

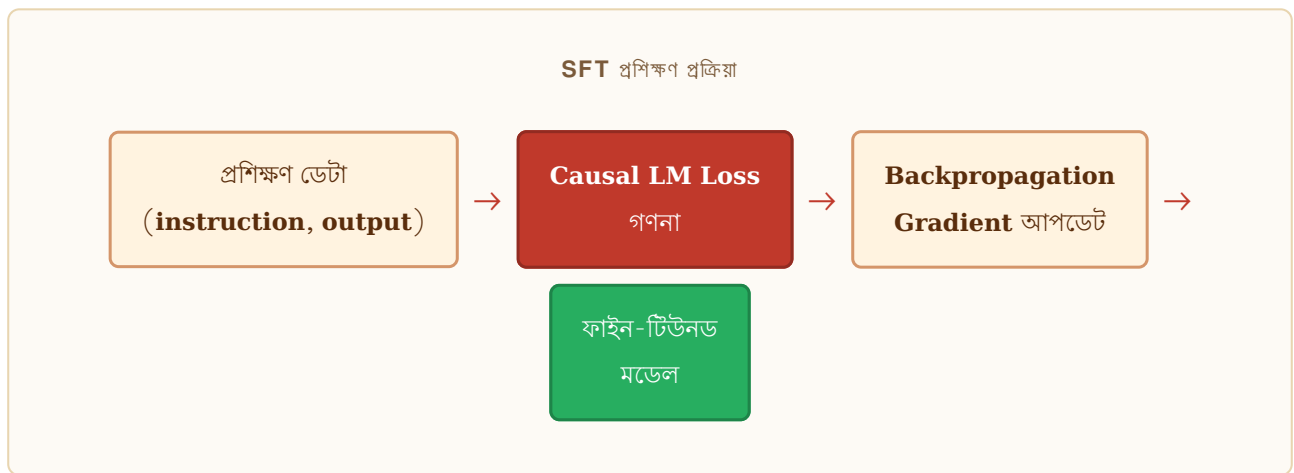
- সাধারণ কাজ: ১,০০০–৫,০০০ উদাহরণ যথেষ্ট
- জটিল ডোমেইন: ১০,০০০–৫০,০০০ উদাহরণ
- চ্যাটবট: ৫,০০০–২০,০০০ কথোপকথন
- কোড জেনারেশন: ৫০,০০০+ উদাহরণ ভালো

Supervised Fine-Tuning (SFT)

ইনপুট-আউটপুট জোড়া দিয়ে মডেল শেখানো

SFT-এর মূল ধারণা

Supervised Fine-Tuning হলো ফাইন-টিউনিংয়ের সবচেয়ে সহজ ও প্রচলিত পদ্ধতি। প্রতিটি প্রশিক্ষণ উদাহরণে একটি ইনপুট এবং একটি কাঙ্ক্ষিত আউটপুট থাকে। মডেল শেখে — এই ধরনের ইনপুটে এই ধরনের আউটপুট দিতে হয়।




```

from transformers import (
    AutoModelForCausalLM, AutoTokenizer,
    TrainingArguments, Trainer, DataCollatorForLanguageModeling
)
from datasets import load_dataset
import torch

## ১. মডেল ও টোকেনাইজার লোড করুন
model_name = "meta-llama/Llama-3.2-1B-Instruct"

tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)

## ২. ডেটা প্রস্তুত করুন
def format_prompt(example):
    text = f"""### নির্দেশনা:
{example['instruction']}

### ইনপুট:
{example['input']}

### উত্তর:
{example['output']}<|end|>"""
    return {"text": text}

def tokenize(example):
    return tokenizer(
        example["text"],
        truncation=True,
        max_length=512,
        padding="max_length"
    )

dataset = load_dataset("json", data_files="train.jsonl")["train"]
dataset = dataset.map(format_prompt).map(tokenize)

## ৩. ট্রেনিং আর্গুমেন্ট
args = TrainingArguments(
    output_dir="./sft_output",
    num_train_epochs=3,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,    # effective batch = 16

```

```

learning_rate=2e-5,
warmup_ratio=0.03,
lr_scheduler_type="cosine",
logging_steps=10,
save_steps=100,
fp16=True, # মেমরি সঞ্চয়
report_to="none",
)

## 8. Trainer তৈরি 3 টুক
trainer = Trainer(
    model=model,
    args=args,
    train_dataset=dataset,
    data_collator=DataCollatorForLanguageModeling(
        tokenizer=tokenizer, mlm=False
    ),
)

trainer.train()
trainer.save_model("./final_model")
print("ফাইন-টিউনিং সম্পন্ন!")

```

গুরুত্বপূর্ণ হাইপারপ্যারামিটার

প্যারামিটার	প্রস্তাবিত মান	প্রভাব
learning_rate	1e-5 – 5e-5	শেখার গতি — খুব বড় হলে মডেল নষ্ট
epochs	1–5	বেশি হলে overfitting
batch_size	4–32	GPU মেমরি অনুযায়ী
max_seq_length	512–2048	দীর্ঘ উত্তরের জন্য বেশি
warmup_ratio	0.03–0.1	শুরুতে lr ধীরে বাড়ানো
weight_decay	0.01–0.1	regularization

CATASTROPHIC FORGETTING

পুরো মডেল ফাইন-টিউন করলে মডেল পুরনো জ্ঞান ভুলে যেতে পারে। সমাধান: খুব কম learning rate ব্যবহার করুন, অথবা LoRA ব্যবহার করুন (পরবর্তী অধ্যায়)।

LoRA ও QLoRA

কম GPU মেমরিতে উচ্চমানের ফাইন-টিউনিং

LoRA — Low-Rank Adaptation

LoRA একটি যুগান্তকারী পদ্ধতি যা ২০২১ সালে Microsoft গবেষকরা প্রস্তাব করেন। ধারণাটি সহজ: মডেলের মূল weight পরিবর্তন না করে, দুটো ছোট matrix যোগ করা হয়।

LoRA-র গাণিতিক ধারণা

$$\begin{array}{|c|} \hline \mathbf{W} \text{ (মূল ওজন)} \\ \hline 1000 \times 1000 \\ \hline \text{জমানো, অপরিবর্তিত} \\ \hline \end{array} + \begin{array}{|c|} \hline \mathbf{A} \times \mathbf{B} \\ \hline 1000 \times 8 \times 8 \times 1000 \\ \hline \text{শুধু এটি শেখা হয়} \\ \hline \end{array} = \begin{array}{|c|} \hline \mathbf{W} + \Delta \mathbf{W} \\ \hline \text{ফাইন-টিউনড আচরণ} \\ \hline \end{array}$$

মূল প্যারামিটার: 1,000,000 | LoRA প্যারামিটার: 16,000 | সাশ্রয়: 98.4%

| LoRA + QLoRA কোড

```

from transformers import AutoModelForCausalLM, BitsAndBytesConfig
from peft import LoraConfig, get_peft_model, TaskType
from trl import SFTTrainer, SFTConfig
import torch

```

```

## ধাপ ১: ৪-বিট কোয়ান্টাইজেশন কনফিগ (QLoRA)

```

```

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",          # NormalFloat4
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True,    # অরু মেমরি সঞ্চয়
)

```

```

## ধাপ ২: মডেল লোড (৪-বিটে)

```

```

model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3.2-7B",
    quantization_config=bnb_config,
    device_map="auto"
)
model.config.use_cache = False

```

```

## ধাপ ৩: LoRA কনফিগ

```

```

lora_config = LoraConfig(
    r=16,                      # rank — ছোট হলে কম প্যারামিটার
    lora_alpha=32,             # scaling: alpha/r
    target_modules=[           # কোন লেয়ারে LoRA যোগ করবেন
        "q_proj", "k_proj",
        "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj"
    ],
    lora_dropout=0.05,
    bias="none",
    task_type=TaskType.CAUSAL_LM
)

```

```

## ধাপ ৪: মডেলে LoRA যোগ করুন

```

```

model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
# trainable params: 83,886,080 || all params: 7.24B (1.16%)

```

```

## ধাপ ৫: SFTTrainer দিয়ে ট্রেন

```

```

trainer = SFTTrainer(
    model=model,
    args=SFTConfig(
        output_dir="./qlora_output",
        num_train_epochs=3,
        per_device_train_batch_size=4,
        gradient_accumulation_steps=4,

```

```

        learning_rate=2e-4,          # LoRA-তে বড় lr ব্যবহার করা যায়
        max_seq_length=1024,
        dataset_text_field="text",
        fp16=True,
    ),
    train_dataset=dataset,
    tokenizer=tokenizer,
)
trainer.train()

## ধাপ ১: মডেল সেভ করুন
model.save_pretrained("./lora_adapter") # পুরনো LoRA অ্যাডাপ্টার

```

LoRA মডেল ব্যবহার করা

```

lora_inference.py

from peft import AutoPeftModelForCausalLM
from transformers import AutoTokenizer

# LoRA মডেল লোড করুন
model = AutoPeftModelForCausalLM.from_pretrained("./lora_adapter")
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-3.2-7B")

# অথবা base model-এ মার্জ করুন (ডিপ্লয়মেন্টের জন্য)
merged_model = model.merge_and_unload()
merged_model.save_pretrained("./merged_model")

```

98%

প্যারামিটার সাশ্রয়

~6GB

7B মডেল QLoRA-তে

2x

দ্রুত ট্রেনিং

~100%

মান বজায় থাকে

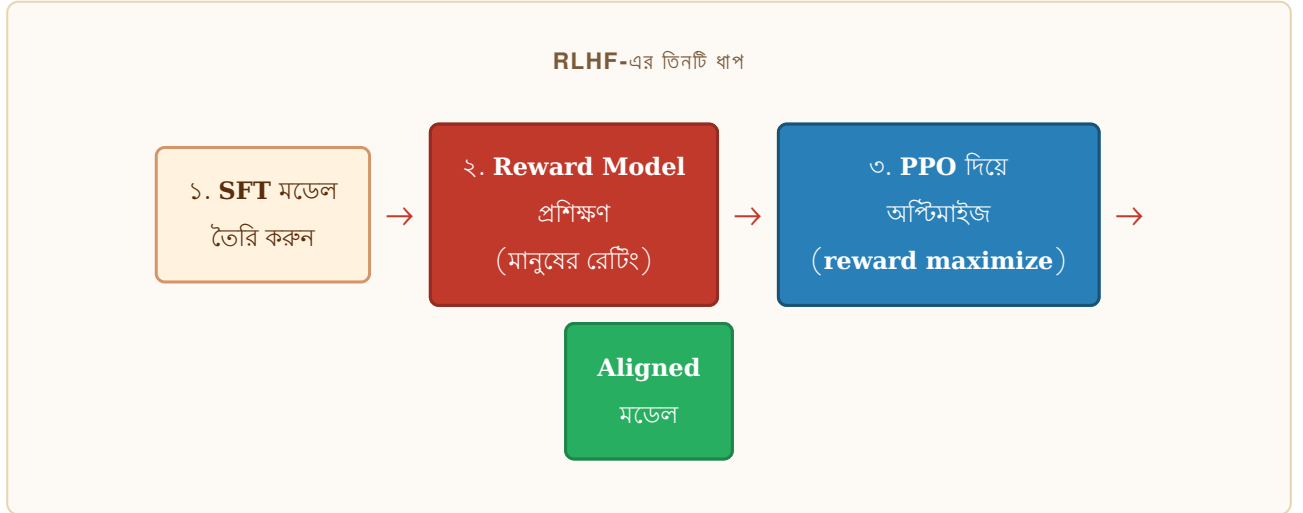
RLHF ও DPO

মানুষের পছন্দ থেকে শেখা — Alignment কৌশল

Alignment সমস্যা কী?

SFT-এর পরে মডেল ইন্সট্রাকশন ফলো করতে পারে, কিন্তু সে সবসময় "helpful, harmless, honest" হয় না। হয়তো ক্ষতিকর বিষয়ে সাহায্য করে, বা মিথ্যা বলে। এই সমস্যার সমাধান হলো Alignment।

RLHF — Reinforcement Learning from Human Feedback



RLHF খুব কার্যকর কিন্তু জটিল। তিনটি আলাদা মডেল, অস্থির PPO ট্রেনিং, এবং বিশাল কম্পিউটেশন দরকার। OpenAI এটি ব্যবহার করে InstructGPT ও ChatGPT তৈরি করেছে।

DPO — Direct Preference Optimization

২০২৩ সালে Stanford গবেষকরা DPO প্রস্তাব করেন। এটি RLHF-এর সরলীকৃত রূপ — Reward Model ছাড়াই "পছন্দের" উত্তর থেকে সরাসরি শেখা যায়।

```

from trl import DPOTrainer, DPOConfig
from datasets import Dataset

## DPO ডেটাসেট ফরম্যাট
# প্রতিটি উদাহরণে: prompt, chosen (ভালো), rejected (খারাপ)
dpo_data = [
    {
        "prompt": "বাংলাদেশের রাজধানী কী?",
        "chosen": "বাংলাদেশের রাজধানী ঢাকা। এটি দেশের বৃহত্তম শহর...",
        "rejected": "রাজধানী হলো একটি শহর যেখানে সরকার থাকে "
    },
]
dataset = Dataset.from_list(dpo_data)

## DPO কনফিগ 3 ট্রেনিং
dpo_config = DPOConfig(
    output_dir="./dpo_output",
    num_train_epochs=1,
    per_device_train_batch_size=2,
    learning_rate=5e-7,          # DPO-তে খুব ছোট lr
    beta=0.1,                   # KL divergence penalty
    max_length=1024,
    max_prompt_length=512,
)

trainer = DPOTrainer(
    model=sft_model,
    ref_model=ref_model,        # মূল SFT মডেল (অপরিবর্তিত)
    args=dpo_config,
    train_dataset=dataset,
    tokenizer=tokenizer,
)
trainer.train()

```

RLHF-এর সুবিধা/অসুবিধা

- ▶ সর্বোচ্চ মানের alignment
- ▶ তিনটি মডেল দরকার
- ▶ PPO ট্রেনিং অস্থির
- ▶ বিশাল কম্পিউট দরকার

DPO-র সুবিধা (প্রস্তাবিত)

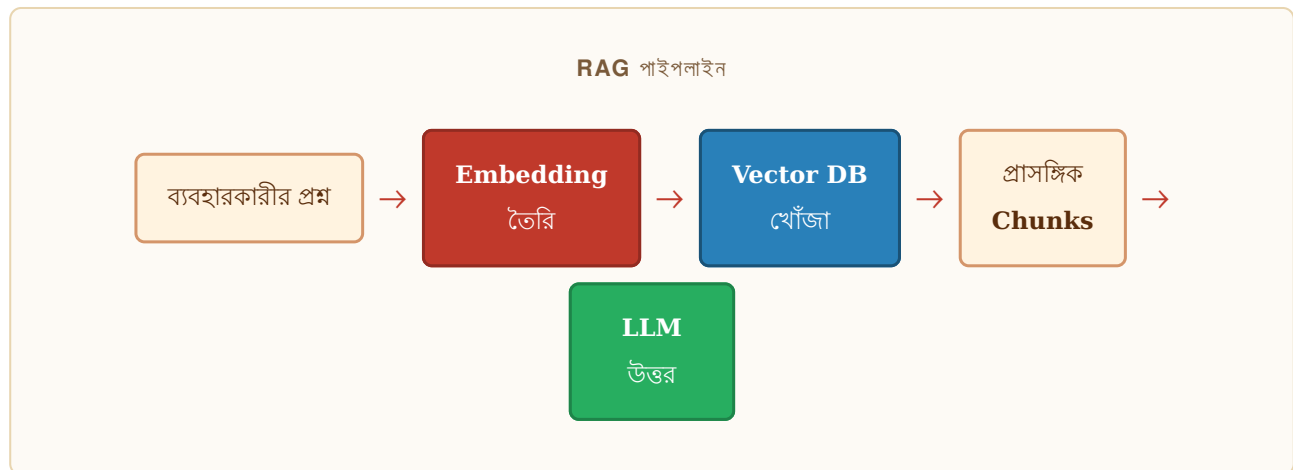
- ▶ Reward model দরকার নেই
- ▶ সহজ ও স্থিতিশীল
- ▶ কম মেমরি ও সময়
- ▶ RLHF-এর কাছাকাছি মান

RAG — Retrieval Augmented Generation

বাইরের জ্ঞানভাণ্ডার থেকে তথ্য নিয়ে উত্তর তৈরি

RAG কেন দরকার?

LLM-এর একটি বড় সমস্যা হলো knowledge cutoff — ট্রেনিং-এর পরের ঘটনা সে জানে না। RAG এই সমস্যার সমাধান করে। মডেল প্রথমে একটি ডকুমেন্ট ডেটাবেস থেকে প্রাসঙ্গিক তথ্য খুঁজে বের করে, তারপর সেই তথ্য ব্যবহার করে উত্তর দেয়।



RAG সিস্টেম তৈরি

rag_system.py

```
from langchain.document_loaders import DirectoryLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import Chroma
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain.chains import RetrievalQA
from langchain_community.llms import Ollama

## ১. ডকুমেন্ট লোড করুন
loader = DirectoryLoader("./docs", glob="**/*.pdf")
documents = loader.load()

## ২. ছোট ছোট Chunk-এ ভাগ করুন
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500, chunk_overlap=50
)
chunks = splitter.split_documents(documents)

## ৩. Embedding তৈরি ও Vector DB-তে স্টোরেজ
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/paraphrase-multilingual-mpnet-base-v2"
)
vectordb = Chroma.from_documents(chunks, embeddings, persist_directory="./chroma_db")

## ৪. RAG Chain তৈরি করুন
llm = Ollama(model="llama3.2") # বা যেকোনো LLM
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=vectordb.as_retriever(search_kwargs={"k": 3}),
    return_source_documents=True
)

## ৫. প্রশ্ন করুন
result = qa_chain("বাংলাদেশের GDP কত?")
print(result["result"])
```

RAG বনাম Fine-Tuning

RAG বেছে নিন যখন

- ▶ তথ্য ঘন ঘন আপডেট হয়
- ▶ সোর্স উল্লেখ করতে হবে
- ▶ নির্দিষ্ট ডকুমেন্ট থেকে উত্তর
- ▶ দ্রুত শুরু করতে চান
- ▶ বড় জ্ঞানভাণ্ডার আছে

Fine-Tuning বেছে নিন যখন

- ▶ মডেলের আচরণ বদলাতে হবে
- ▶ নির্দিষ্ট ফরম্যাট শেখাতে হবে
- ▶ ডোমেইন-নির্দিষ্ট ভাষা
- ▶ কম latency দরকার
- ▶ ডেটা গোপন রাখতে হবে

মডেল মূল্যায়ন

কীভাবে বুঝবেন মডেল ভালো হয়েছে — Metrics ও Benchmarks

মূল্যায়ন কেন জরুরি?

ফাইন-টিউনিংয়ের পরে জানতে হবে — মডেল আসলেই উন্নত হয়েছে কিনা। শুধু training loss কমা যথেষ্ট নয়। সঠিক মূল্যায়ন ছাড়া আপনি জানবেন না মডেল ব্যর্থ হচ্ছে কোথায়।

স্বয়ংক্রিয় মেট্রিক

মেট্রিক	কাজ	ব্যাখ্যা
Perplexity	ভাষা মডেলিং	কম = ভালো। মডেল কতটা "নিশ্চিত"
BLEU	অনুবাদ	রেফারেন্সের সাথে n-gram মিল
ROUGE-L	সারাংশ	দীর্ঘতম সাধারণ ক্রম খোঁজা
BERTScore	সাধারণ	Embedding-এর cosine similarity
Exact Match	প্রশ্নোত্তর	হুবহু মিল আছে কিনা
Pass@k	কোড	k চেষ্টায় সঠিক কোড পাওয়ার হার

LLM-as-Judge পদ্ধতি

```
llm_judge.py

import anthropic
import json

client = anthropic.Anthropic()

def evaluate_response(question: str, response: str, reference: str) -> dict:
    judge_prompt = f"""অপনি একজন নিরপেক্ষ মূল্যায়নকারী।

    প্রশ্ন: {question}
    মডেলের উত্তর: {response}
    সঠিক/আদর্শ উত্তর: {reference}

    নিচের মানদণ্ডে ১-৫ স্কেলে মূল্যায়ন করুন:
    - সঠিকতা (Accuracy): তথ্য কতটা সঠিক
    - সম্পূর্ণতা (Completeness): প্রশ্নের সব দিক কভার হয়েছে কিনা
    - ভাষার মান (Language): বাংলা কতটা প্রাক্কর ৩ সঠিক

    গুঁড়ু JSON ফরম্যাটে উত্তর দিন:
    {{ "accuracy": 4, "completeness": 3, "language": 5, "overall": 4, "feedback": "..."} }"""

    result = client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=300,
        messages=[{"role": "user", "content": judge_prompt}]
    )
    return json.loads(result.content[0].text)

# ব্যবহার:
score = evaluate_response(
    question="পদ্মা স্রোতের দৈর্ঘ্য কত?",
    response="পদ্মা স্রোত ৬.১৫ কিলোমিটার দীর্ঘ।",
    reference="পদ্মা স্রোতের মূল দৈর্ঘ্য ৬.১৫ কিমি, মোট দৈর্ঘ্য ৯.৬৩ কিমি "
)
print(score)
```

বিখ্যাত বেঞ্চমার্ক

- **MMLU (Massive Multitask Language Understanding)**: ৫৭টি বিষয়ে সাধারণ জ্ঞান পরীক্ষা
- **HumanEval**: Python কোড সমস্যা সমাধানের দক্ষতা পরিমাপ
- **MT-Bench**: মাল্টি-টার্ন কথোপকথনের মান মূল্যায়ন
- **HELM**: Stanford-এর সামগ্রিক LLM মূল্যায়ন ফ্রেমওয়ার্ক

- **BengaliGLUE:** বাংলা ভাষার বিভিন্ন NLP কাজের বেঞ্চমার্ক

ডিপ্লয়মেন্ট ও প্রোডাকশন

ফাইন-টিউনড মডেলকে বাস্তবে ব্যবহারযোগ্য করা

ডিপ্লয়মেন্ট পদ্ধতির তুলনা

পদ্ধতি	গতি	সহজতা	GPU	ব্যবহার
vLLM	সর্বোচ্চ	মধ্যম	হ্যাঁ	প্রোডাকশন API
TGI (HF)	উচ্চ	সহজ	হ্যাঁ	Hugging Face
Ollama	মধ্যম	সহজতম	ঐচ্ছিক	লোকাল ডেভেলপমেন্ট
llama.cpp	কম	মধ্যম	না	CPU সার্ভার
FastAPI+HF	মধ্যম	সহজ	হ্যাঁ	কাস্টম API

vLLM দিয়ে প্রোডাকশন API

```
vllm_deploy.sh + api_client.py

## সার্ভার চালান (Terminal)
$ pip install vllm
$ python -m vllm.entrypoints.openai.api_server \
  --model ./finetuned_model \
  --host 0.0.0.0 \
  --port 8000 \
  --max-model-len 4096 \
  --gpu-memory-utilization 0.9

## Python Client
from openai import OpenAI

client = OpenAI(
    base_url="http://localhost:8000/v1",
    api_key="token-no-needed"
)

response = client.chat.completions.create(
    model="finetuned_model",
    messages=[
        {"role": "system", "content": "আপনি একটি সহায়ক AI।"},
        {"role": "user", "content": "বাংলাদেশ সম্পর্কে বলুন"}
    ],
    max_tokens=512,
    temperature=0.7
)
print(response.choices[0].message.content)
```

মডেল কোয়ান্টাইজেশন

প্রোডাকশনে ব্যবহারের আগে মডেল ছোট করা (quantize) যায় — এতে গতি বাড়ে, মেমরি কমে।

quantize_to_gguf.py

```
# Unsloth দিয়ে GGUF ফরম্যাটে রূপান্তর
from unsloth import FastLanguageModel

model, tokenizer = FastLanguageModel.from_pretrained(
    "./finetuned_model"
)

# GGUF ফরম্যাটে স্টেভ করুন (Ollama-র জন্য)
model.save_pretrained_gguf(
    "bangla_model_gguf",
    tokenizer,
    quantization_method="q4_k_m" # 4-bit, ভালো মানের
)

# Ollama দিয়ে চালানোর জন্য Modelfile তৈরি করুন
modelfile = """FROM ./bangla_model_gguf/model.gguf
PARAMETER temperature 0.7
PARAMETER num_ctx 2048
SYSTEM আপনি একটি বাংলা ভাষার সহায়ক AI: """

with open("Modelfile", "w") as f:
    f.write(modelfile)

# চালান: ollama create bangla-bot -f Modelfile
# ব্যবহার: ollama run bangla-bot "বাংলাদেশ কী?"
```

Model Merging — একাধিক মডেল একত্রিত করা

দুটো ভিন্ন কাজে ফাইন-টিউনড মডেল মার্জ করে একটি শক্তিশালী মডেল তৈরি করা সম্ভব। যেমন: বাংলা ভাষার মডেল + কোড লেখার মডেল = বাংলায় কোড ব্যাখ্যা করার মডেল।

```
merge_config.yaml

# mergekit ব্যবহার করুন: pip install mergekit
models:
  - model: ./bangla-finetuned
    parameters:
      weight: 0.6
  - model: ./code-finetuned
    parameters:
      weight: 0.4
merge_method: ties          # TIES মার্জিং পদ্ধতি
base_model: meta-llama/Llama-3.2-1B
parameters:
  normalize: true
  int8_mask: true

# কমান্ড: mergekit-yaml merge_config.yaml ./merged_model
```

Unsloth — ২x দ্রুত ফাইন-টিউনিং

unsloth_training.py

```
# pip install "unsloth[colab-new] @ git+https://github.com/unslothai/unsloth.git"
from unsloth import FastLanguageModel
import torch

## মডেল লোড — ২x দ্রুত, ৮০% কম মেমরি
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="unsloth/Llama-3.2-1B-Instruct-bnb-4bit",
    max_seq_length=2048,
    dtype=None,
    load_in_4bit=True,
)

model = FastLanguageModel.get_peft_model(
    model,
    r=16, lora_alpha=16,
    target_modules=["q_proj", "k_proj", "v_proj",
                  "o_proj", "gate_proj", "up_proj", "down_proj"],
    lora_dropout=0, # Unsloth-এ ০ হলে দ্রুত
    bias="none",
    use_gradient_checkpointing="unsloth", # বিশেষ অপটিমাইজেশন
    random_state=42,
)

model.print_trainable_parameters()
# Unsloth: 2x faster, 80% less memory than standard QLoRA
```


Multi-GPU ফাইন-টিউনিং

```
multi_gpu.sh

# DeepSpeed ZeRO-3 দিয়ে ৪টি GPU-তে
$ accelerate launch \
  --config_file deepspeed_zero3.yaml \
  --num_processes 4 \
  sft_training.py

# deepspeed_zero3.yaml (অংকিত)
compute_environment: LOCAL_MACHINE
distributed_type: DEEPSPEED
deepspeed_config:
  zero_stage: 3
  offload_optimizer_device: cpu
  offload_param_device: cpu
```

2x

Unsloth গতি বৃদ্ধি

80%

কম GPU মেমরি

4x

4-GPU গতি বৃদ্ধি

TIES

সেরা Merge পদ্ধতি

বাংলাদেশ ও বাংলা ভাষার প্রেক্ষাপটে LLM ব্যবহার

বাংলা ভাষার বিশেষ চ্যালেঞ্জ

বাংলা পৃথিবীর ৫ম বৃহত্তম ভাষা — ৩০ কোটিরও বেশি মানুষ কথা বলে। তবু বেশিরভাগ LLM বাংলায় দুর্বল, কারণ প্রশিক্ষণ ডেটার বড় অংশ ইংরেজিতে।

বাংলা LLM-এর চ্যালেঞ্জ

- ▶ কম প্রশিক্ষণ ডেটা
- ▶ বেশি টোকেন প্রতি শব্দে
- ▶ জটিল মরফোলজি
- ▶ দুটো লিপি (বাংলা ও ইংরেজি মিশ্রণ)
- ▶ কম ওপেন-সোর্স মডেল
- ▶ ডায়ালেক্ট বৈচিত্র্য

উদীয়মান বাংলা মডেল

- ▶ BanglaBERT (BUET)
- ▶ BanglaT5
- ▶ Bangla-LLaMA
- ▶ ShahiBloom (Meta)
- ▶ MuktiBERT
- ▶ BanglaGPT (উন্নয়নশীল)

বাংলাদেশে LLM-এর ব্যবহার ক্ষেত্র

- **কৃষি পরামর্শ:** কৃষকদের বাংলায় ফসল রোগ নির্ণয় ও সমাধান
- **স্বাস্থ্যসেবা:** গ্রামীণ জনগণের জন্য প্রাথমিক স্বাস্থ্য পরামর্শ
- **শিক্ষা:** ছাত্রদের জন্য বাংলায় ব্যাখ্যা ও প্রশ্নোত্তর
- **আইনি সহায়তা:** সরল ভাষায় আইনি অধিকার বোঝানো
- **সরকারি সেবা:** e-সেবায় নাগরিকদের সহায়তা
- **সংবাদ মাধ্যম:** স্বয়ংক্রিয় সারাংশ ও অনুবাদ

বাংলা LLM ফাইন-টিউনিং টিপস

● ● ● bangla_specific_tips.py

```
## ১. বাংলা টোকেনাইজার পরীক্ষা করুন
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-3.2-1B")
bn_text = "আমি বাংলাদেশে থাকি "
en_text = "I live in Bangladesh."

print(f"বাংলা টোকেন: {len(tokenizer.encode(bn_text))}") # ~12
print(f"ইংরেজি টোকেন: {len(tokenizer.encode(en_text))}") # ~6

## ২. বাংলার জন্য বেশি max_seq_length রাখুন
SFTConfig(max_seq_length=2048) # ইংরেজির ১-৪ গুণ

## ৩. বাংলা-বক্কু মডেল বেছে নিন
# ডানো বিকল্প:
# - meta-llama/Llama-3.1-8B (ডানো multilingual)
# - Qwen/Qwen2.5-7B-Instruct (বাংলায় ডানো)
# - google/gemma-2-9b-it (multilingual)

## ৪. বাংলা system prompt দিন
SYSTEM = """অপনি একটি মহায়ক AI মহাকারী।
মবময় স্পষ্ট, মহজ বাংলায় উত্তর দিন।
পরিভাষা ব্যবহার করনে মহজ ভাষায় ব্যাখ্যা করুন।"""
```

বাংলা LLM-এর ভবিষ্যৎ

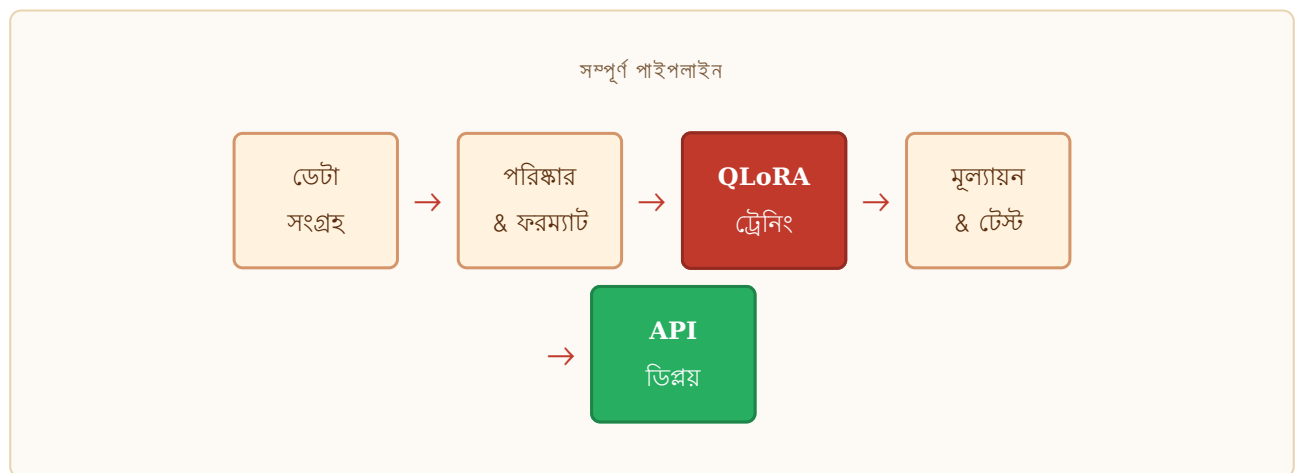
বাংলাদেশের AI গবেষণা দ্রুত বাড়ছে। BUET, NSU, IUB সহ বিভিন্ন বিশ্ববিদ্যালয় বাংলা NLP নিয়ে কাজ করছে। ১০ কোটি+ বাংলাভাষী ইন্টারনেট ব্যবহারকারী থাকায় এই বাজার বিশাল সম্ভাবনাময়।

সম্পূর্ণ প্রজেক্ট

বাংলা কাস্টমার সাপোর্ট চ্যাটবট — শুরু থেকে শেষ

প্রজেক্ট পরিচিতি

এই অধ্যায়ে আমরা একটি বাস্তব প্রজেক্ট সম্পন্ন করব। লক্ষ্য: LLaMA-3.2-1B মডেলকে বাংলাদেশি ই-কমার্স কাস্টমার সাপোর্টের জন্য ফাইন-টিউন করা এবং API হিসেবে চালু করা।



ধাপ ১: ডেটা প্রস্তুতি

step1_data_prep.py

```
import json

# নমুনা ই-কমার্স Q&A ডেটা
training_data = [
    {
        "messages": [
            {"role": "system", "content": "আপনি ShopBD-র কাস্টমার সার্ভিস এজেন্ট।"},
            {"role": "user", "content": "আমার অর্ডার কখন পাব?"},
            {"role": "assistant", "content": "আপনার অর্ডার সাধারণত ৩-৫ কর্মদিবসে পৌঁছাবে..."}
        ]
    },
    {
        "messages": [
            {"role": "system", "content": "আপনি ShopBD-র কাস্টমার সার্ভিস এজেন্ট।"},
            {"role": "user", "content": "পণ্য ফেরত দিতে চাই"},
            {"role": "assistant", "content": "অবশ্যই ডেলিভারির ৭ দিনের মধ্যে ফেরত নেওয়া হয়..."}
        ]
    }
]

# JSONL ফাইলে সেভ করুন
with open("train.jsonl", "w", encoding="utf-8") as f:
    for item in training_data:
        f.write(json.dumps(item, ensure_ascii=False) + "\n")
```

| ধাপ ২: **Unsloth** দিয়ে ট্রেনিং

```

from unsloth import FastLanguageModel
from trl import SFTTrainer, SFTConfig
from datasets import load_dataset

## মডেল লোড
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="unsloth/Llama-3.2-1B-Instruct-bnb-4bit",
    max_seq_length=2048,
    load_in_4bit=True,
)
model = FastLanguageModel.get_peft_model(
    model, r=16, lora_alpha=32,
    target_modules=["q_proj", "v_proj", "k_proj", "o_proj"],
    use_gradient_checkpointing="unsloth",
)

## ডেটাসেট
dataset = load_dataset("json", data_files="train.jsonl")["train"]

## চ্যাট টেমপ্লেট ফরম্যাট
def format_chat(example):
    formatted = tokenizer.apply_chat_template(
        example["messages"],
        tokenize=False,
        add_generation_prompt=False
    )
    return {"text": formatted}

dataset = dataset.map(format_chat)

## ট্রেনিং
trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    train_dataset=dataset,
    args=SFTConfig(
        output_dir="./shopbd_chatbot",
        num_train_epochs=3,
        per_device_train_batch_size=4,
        gradient_accumulation_steps=4,
        learning_rate=2e-4,
        max_seq_length=2048,
        dataset_text_field="text",
        fp16=True,
        logging_steps=10,
    ),
)
trainer.train()

```

```
## GGUF স্নেহ
model.save_pretrained_gguf("shopbd_gguf", tokenizer, quantization_method="q4_k_m")
print("ট্রেনিং সম্পন্ন!")
```

ধাপ ৩: FastAPI দিয়ে API তৈরি

```
step3_api_server.py

from fastapi import FastAPI
from pydantic import BaseModel
from transformers import pipeline
import uvicorn

app = FastAPI(title="ShopBD চ্যাটবট API")

# মডেল লোড করুন
chatbot = pipeline(
    "text-generation",
    model="./shopbd_chatbot",
    max_new_tokens=256
)

class ChatRequest(BaseModel):
    message: str
    history: list = []

@app.post("/chat")
async def chat(request: ChatRequest):
    messages = [
        {"role": "system", "content": "আপনি ShopBD-র সহায়ক"},
        *request.history,
        {"role": "user", "content": request.message}
    ]
    response = chatbot(messages)[0]["generated_text"][-1]["content"]
    return {"response": response}

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

পরবর্তী শেখার পথ

- **Multimodal Fine-tuning:** LLaVA, Qwen-VL দিয়ে ছবি + টেক্সট
- **Constitutional AI:** নিজের নিয়মকানুন থেকে মডেল শেখানো
- **Continual Learning:** মডেলকে নিয়মিত নতুন তথ্য শেখানো

- **Mixture of Experts:** বড় মডেলের উন্নত আর্কিটেকচার
- **Speculative Decoding:** ইনফারেন্স ৩-৪x দ্রুত করার পদ্ধতি
- **Hugging Face Hub:** নিজের মডেল বিশ্বের সাথে শেয়ার করুন

অভিনন্দন! আপনি সম্পূর্ণ বইটি পড়েছেন।

আপনি এখন LLM-এর মূল ধারণা থেকে শুরু করে ফাইন-টিউনিং, মূল্যায়ন, ডিপ্লয়মেন্ট এবং বাস্তব প্রজেক্ট সম্পর্কে বিস্তারিত জানেন। এখন সময় হলো হাতে-কলমে চেষ্টা করার। Google Colab বা Kaggle Notebooks-এ বিনামূল্যে GPU পাবেন — আজই শুরু করুন!

গুরুত্বপূর্ণ রিসোর্স

- **Hugging Face Docs:** huggingface.co/docs — সেরা ডকুমেন্টেশন
- **TRL Library:** github.com/huggingface/trl — SFT, DPO, RLHF
- **Unsloth:** github.com/unslothai/unsloth — দ্রুত ফাইন-টিউনিং
- **LLaMA Factory:** github.com/hiyouga/LLaMA-Factory — GUI সহ
- **Papers With Code:** paperswithcode.com — সর্বশেষ গবেষণাপত্র
- **FastAI Course:** fast.ai — Deep Learning কোর্স (বিনামূল্যে)